NLP 243 Deep Learning for Natural Language Processing

Fall 2024

Outline for Today

- **Continuing:**
  - What is machine learning
    - Reinforcement learning
  - What is deep learning?

- **Preliminaries**
  - PyTorch Basics and Linear Algebra
  - Calculus - Derivatives and Differentiation
  - NLP Basic Terminology

- Readings:
  - Ch1 of NLP with PyTorch
  - Ch2 of Dive into Deep Learning -> The review chapter!

Note: Syllabus Acknowledgement: Due 30 Sept

Outline for Today

- Continuing:
  - What is machine learning
    - Reinforcement learning
  - What is deep learning?
- Preliminaries
  - PyTorch Basics and Linear Algebra
  - Calculus - Derivatives and Differentiation
  - NLP Basic Terminology

- Readings:
  - Ch1 of NLP with PyTorch
  - Ch2 of Dive into Deep Learning -> The review chapter!

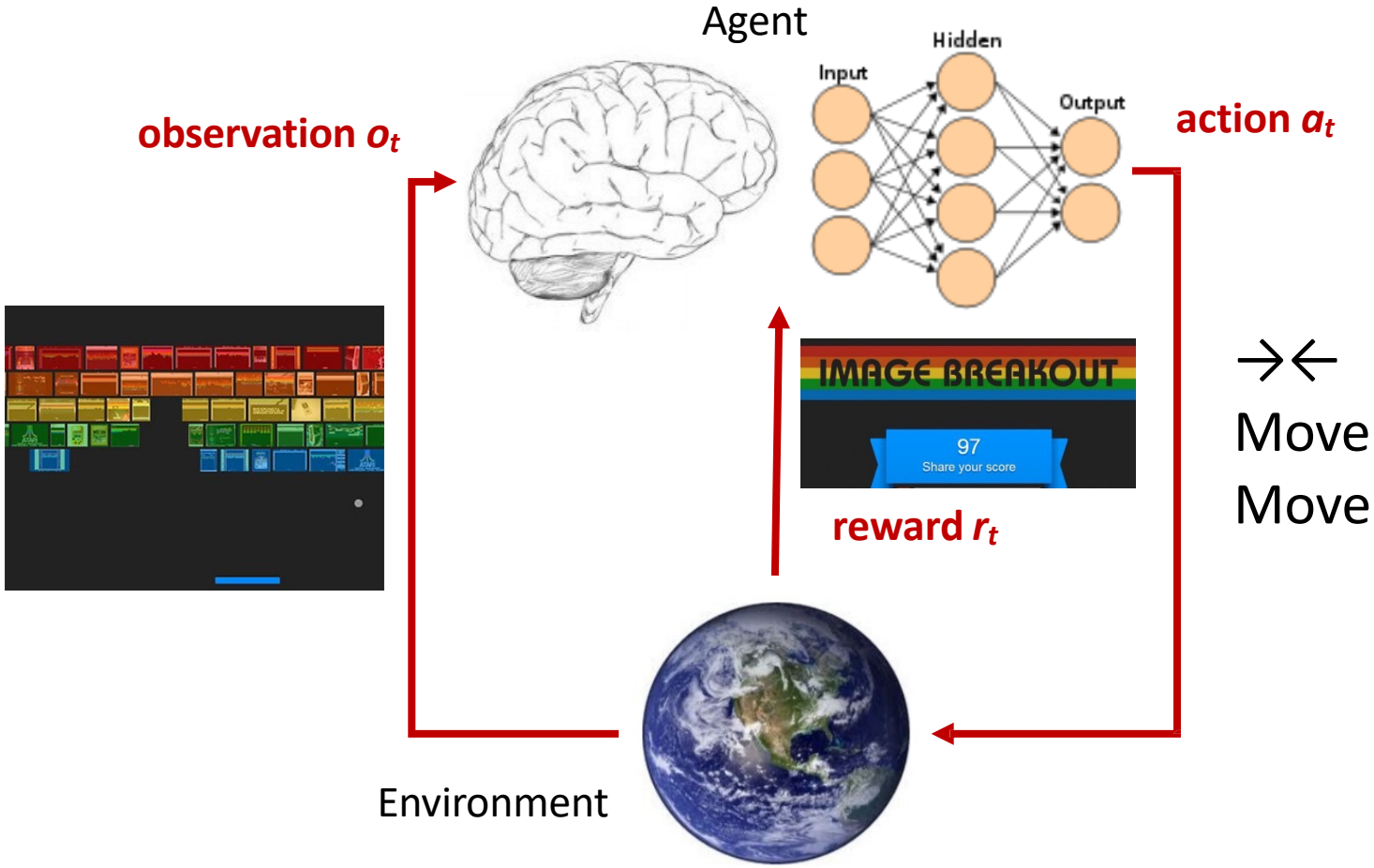
Note: Syllabus Acknowledgement: Due 30 Sept

Reinforcement Learning

- Learning action prediction policies through interactions with an environment over a series of *timesteps*.
  - At each timestep t, the learning agent receives some observation o_t from the environment and must choose an action a_t that is subsequently transmitted back to the environment via some mechanism.
  - Finally, the agent receives a reward r_t from the environment.
  - The agent then receives a subsequent observation, and chooses a subsequent action, and so on.
  - A *policy* is a function that maps from observations to actions.

Agent and Environment

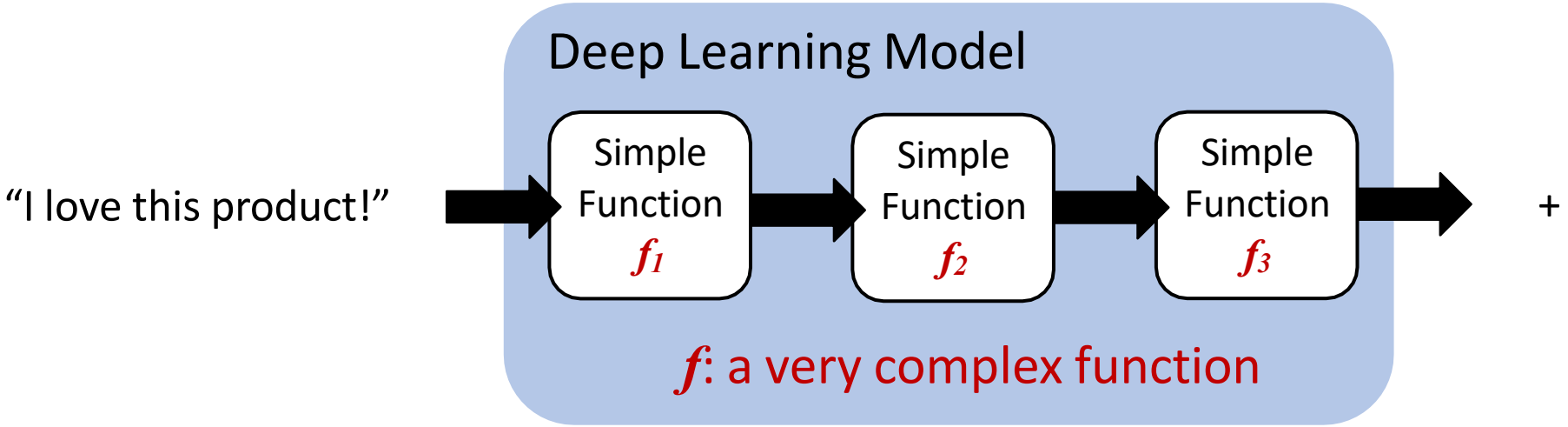
- No labels, just a reward
- Agent can exploit and explore
- More on RL in NLP 244!



→←
MoveRight
MoveLeft

Deep Learning

Stacked Functions Learned by Machine

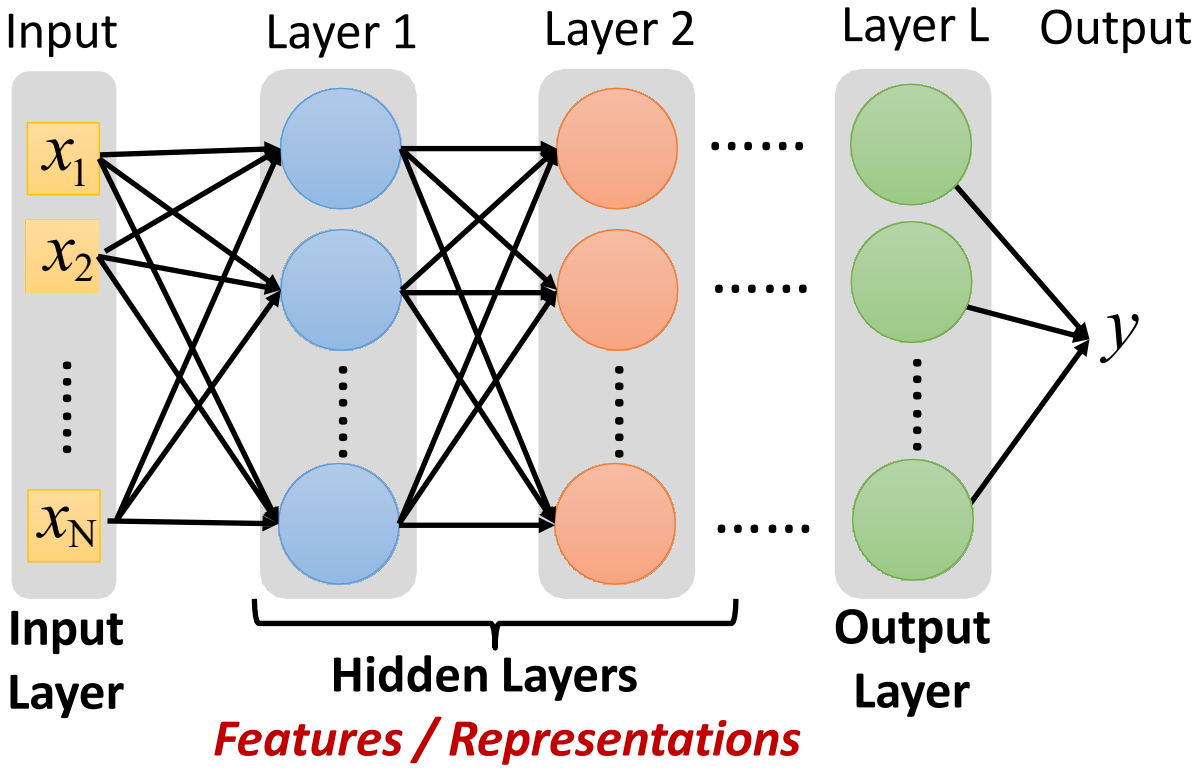


End-to-end training: what each function should do is learned automatically

Deep learning usually refers to *neural network* based model

Stacked Functions Learned by Machine

vector $\boldsymbol{x}$
"I love this product."



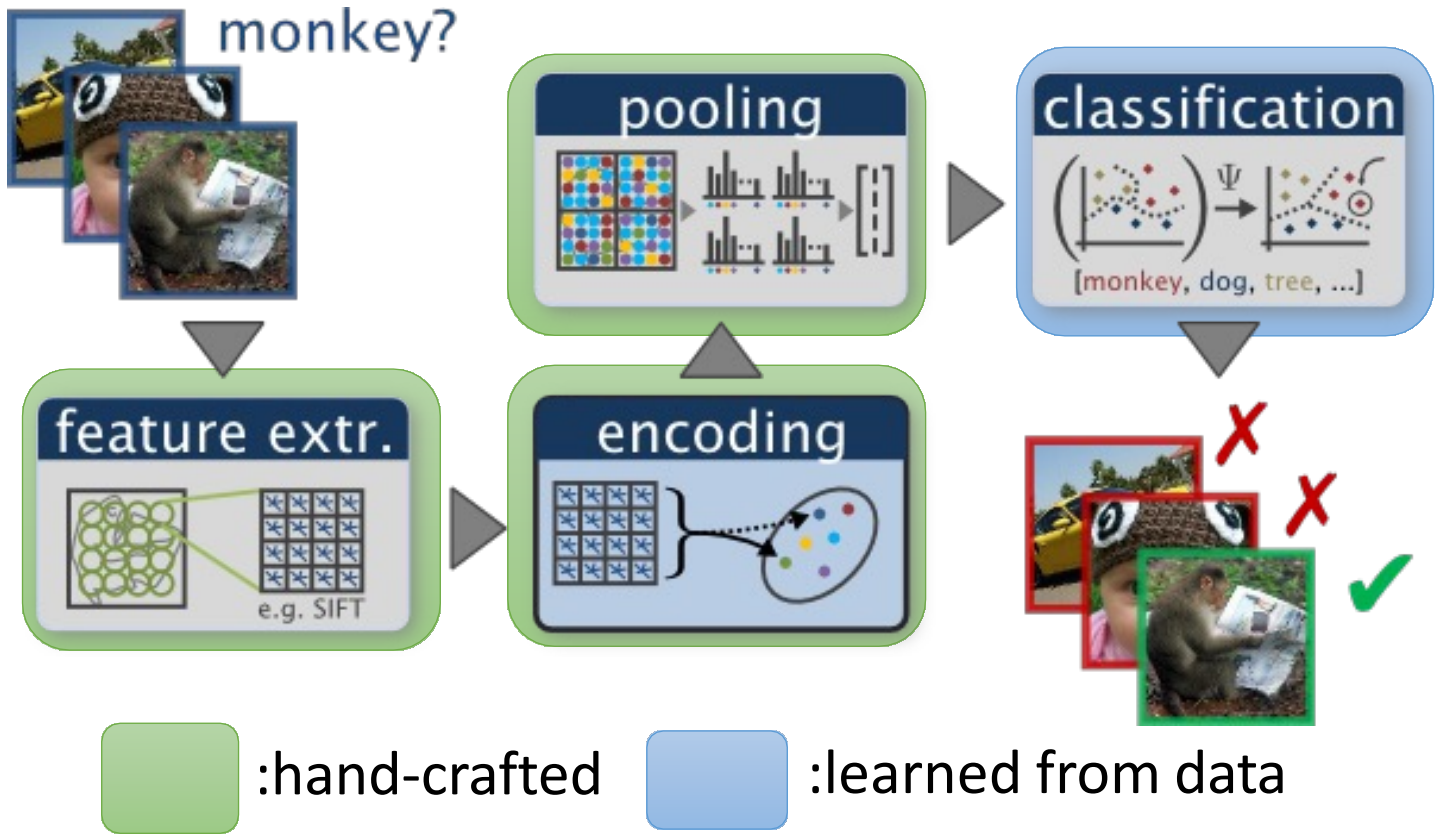
label $\boldsymbol{y}$
+

Representation Learning attempts to learn good features/representations

Deep Learning attempts to learn (multiple levels of) representations and an output

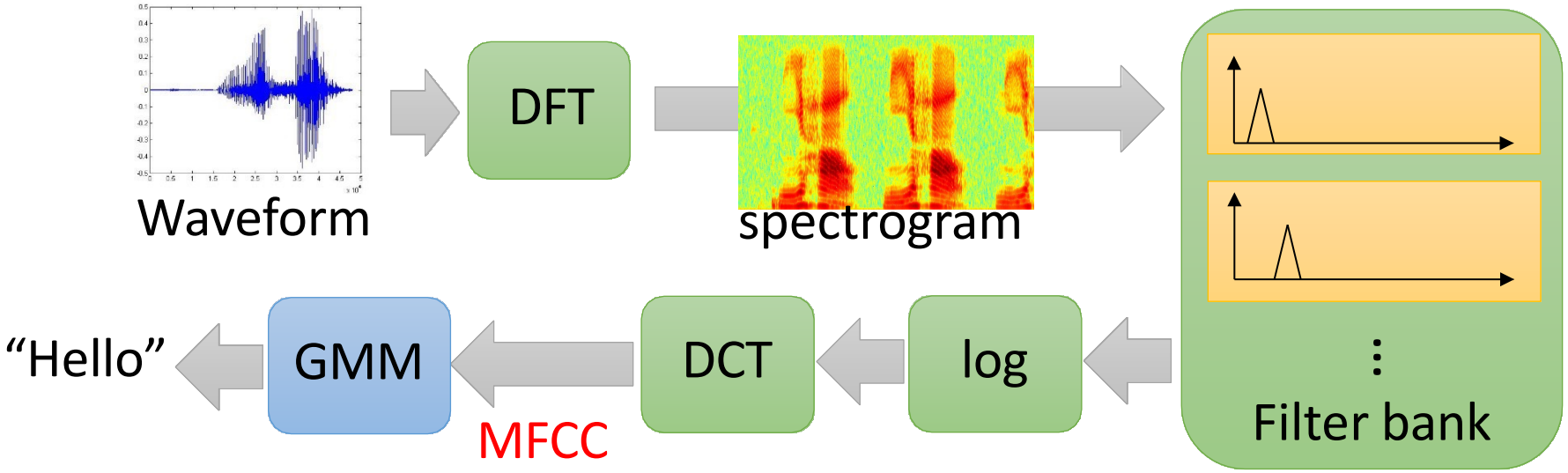
Deep vs. Shallow – Image Recognition

- Shallow Model



Deep vs. Shallow – Speech Recognition

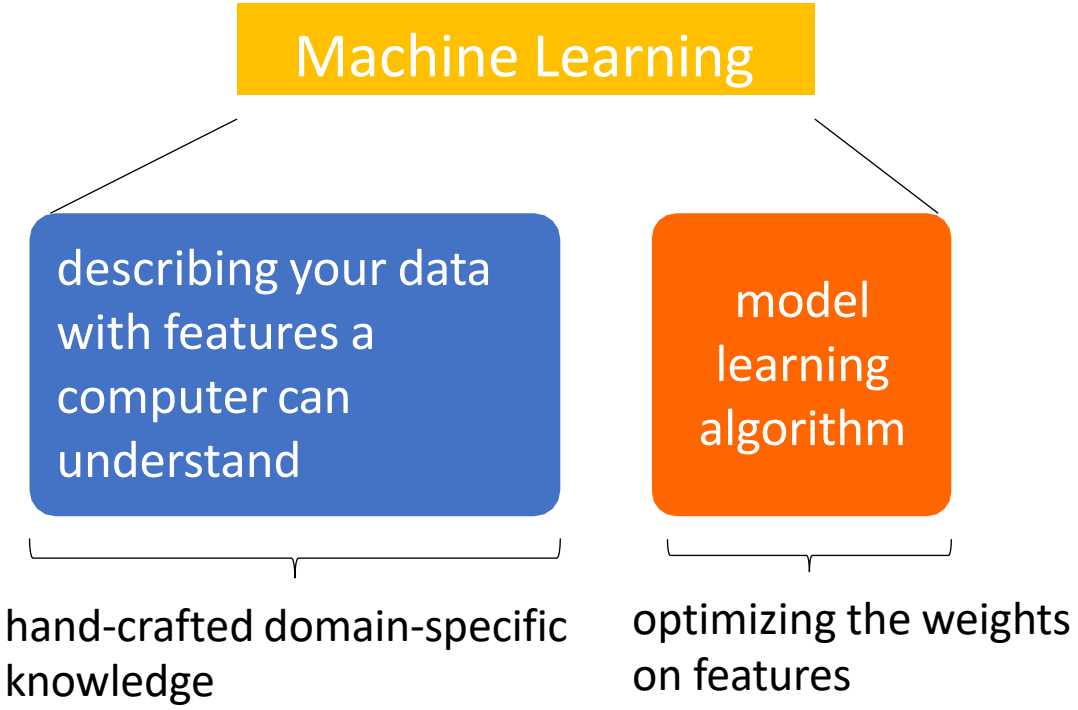
- Shallow Model



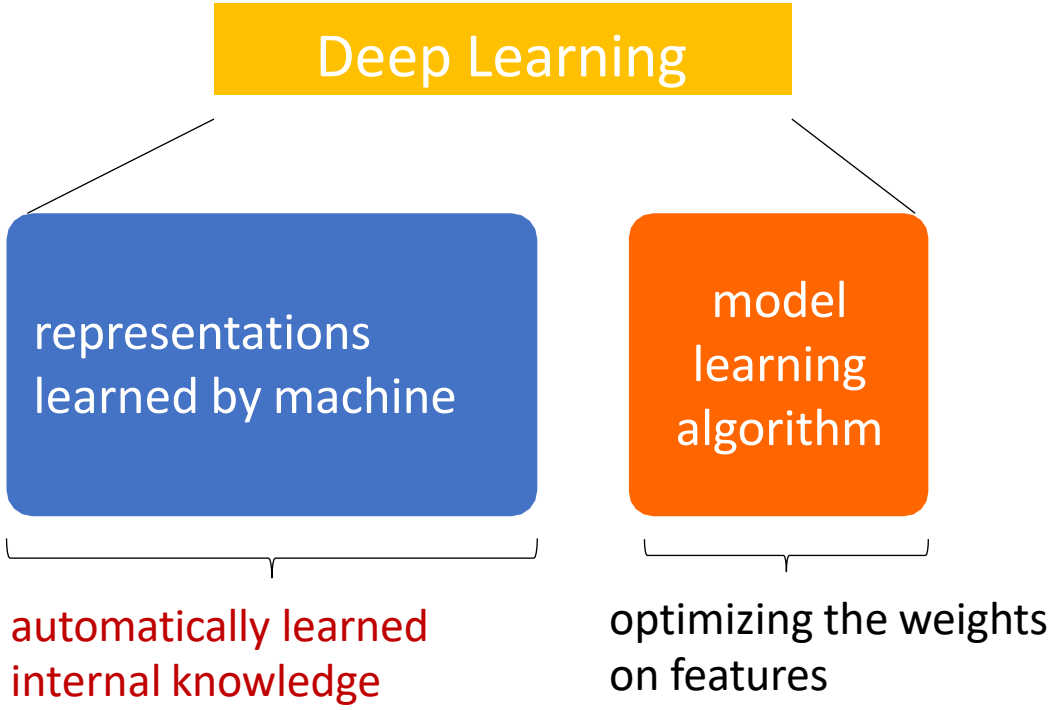
Each box is a simple function in the production line:

:hand-crafted :learned from data

Machine Learning vs. Deep Learning

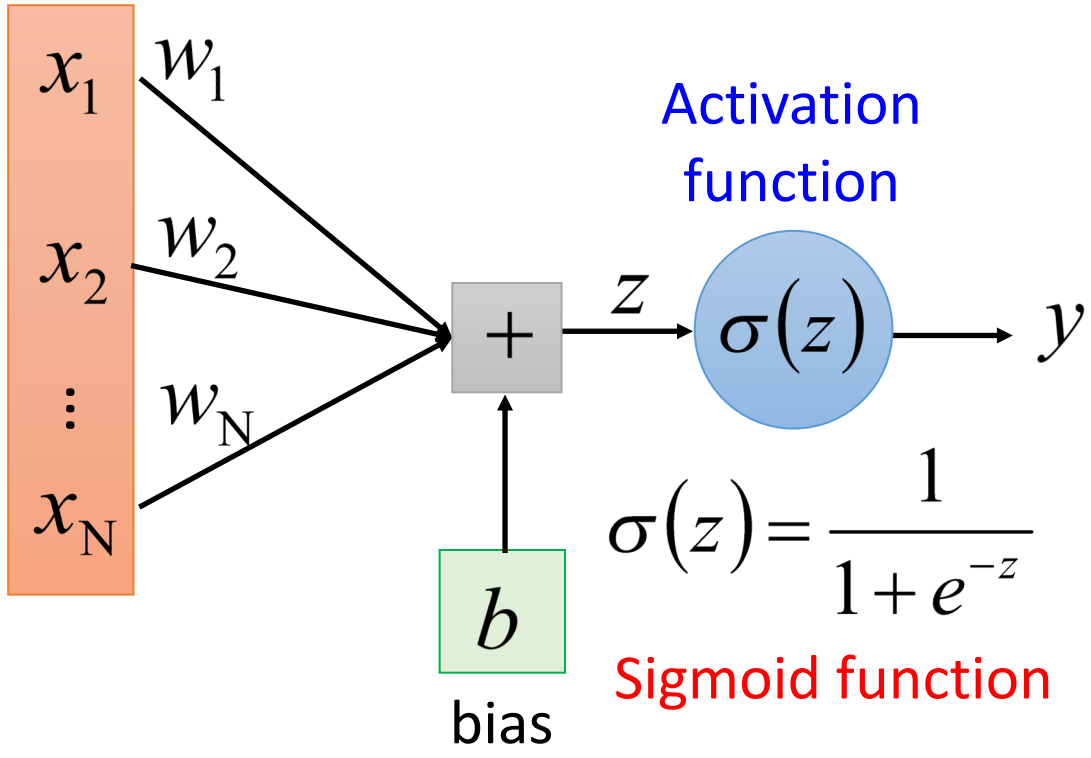


Machine Learning vs. Deep Learning



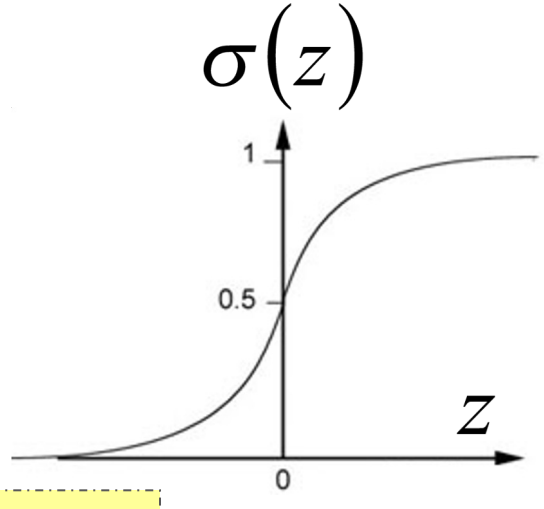
Deep learning usually refers to *neural network* based model

A Single Neuron



$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Sigmoid function



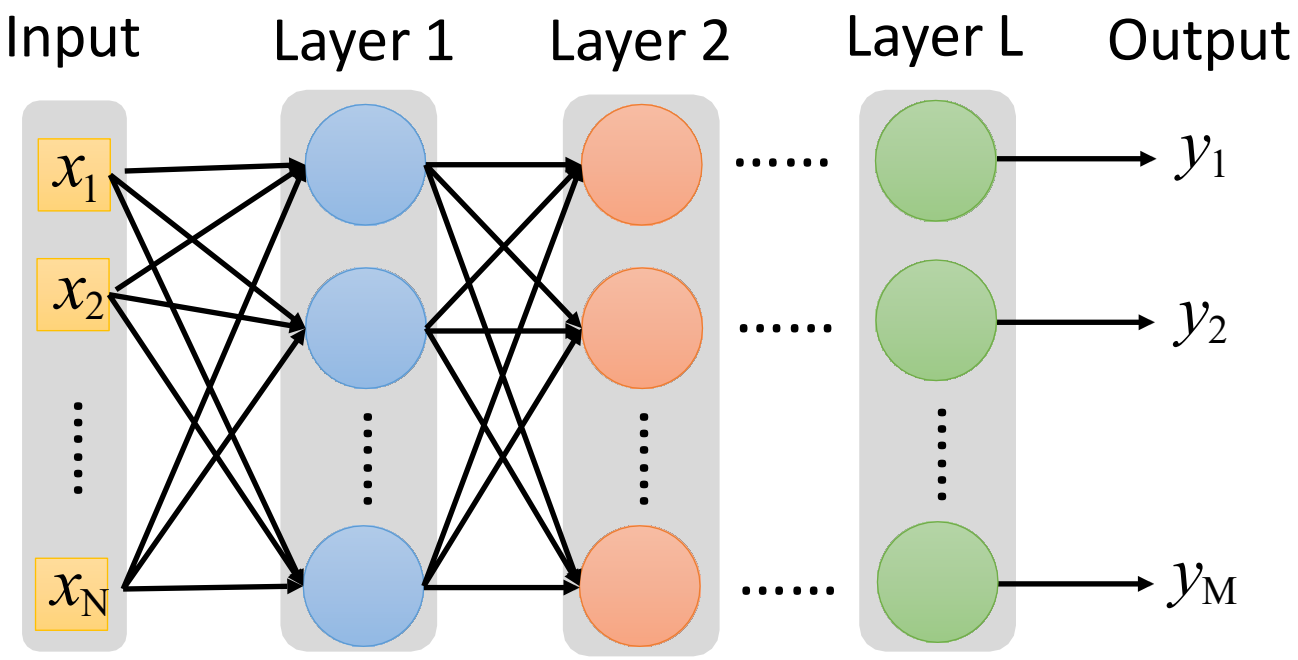
Each neuron is a very simple function

Deep Neural Network

A neural network is a complex function:

$$f : R^N \text{ ® } R^M$$

- Cascading the neurons to form a neural network

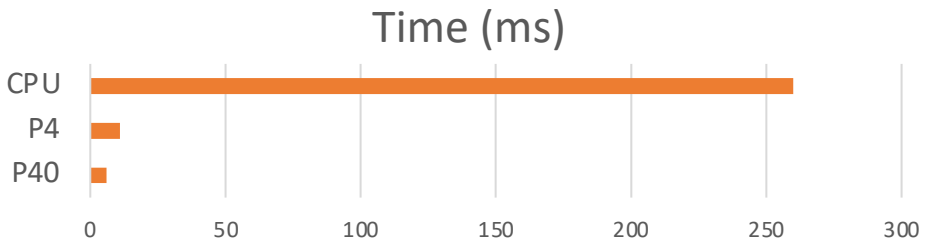


Each layer is a simple function in the production line

Why does speed matter?

- Training time
  - Big data increases the training time
  - Too long training time is not practical

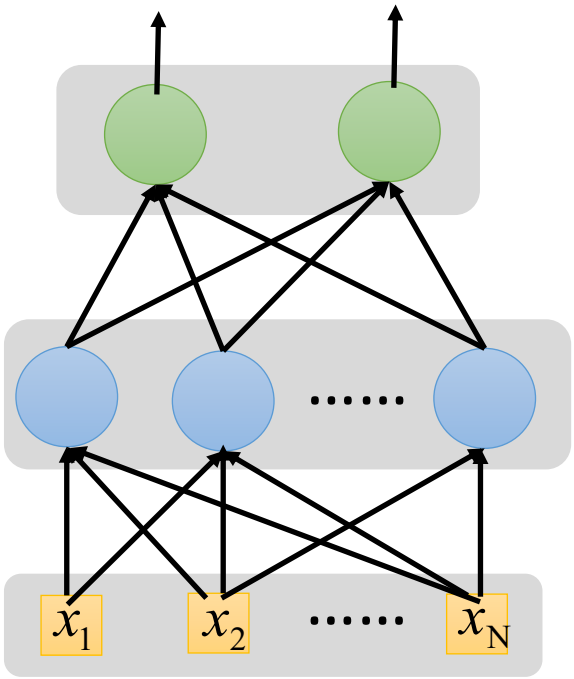
- Inference time
  - Users are not patient to wait for the responses



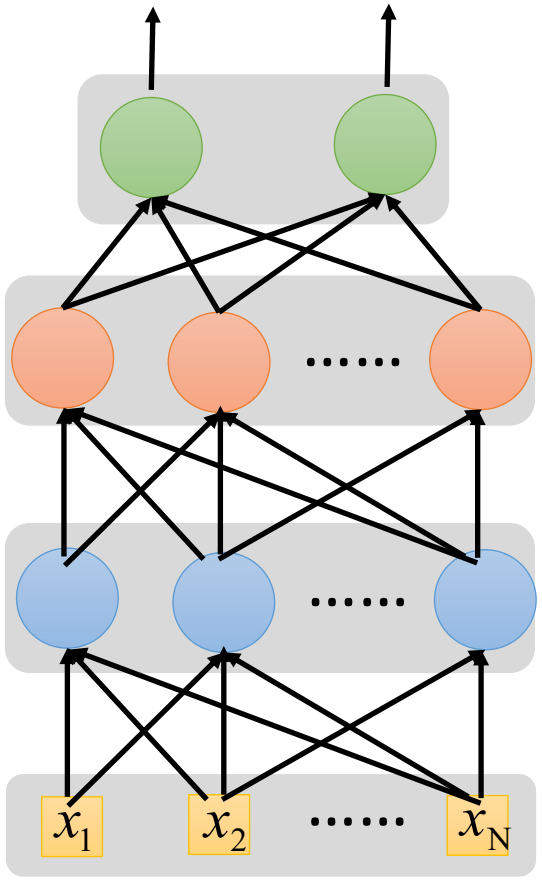
GPU enables the real-world applications using the computational power

Why is deeper better?

- Deeper → More parameters?



Shallow



Deep

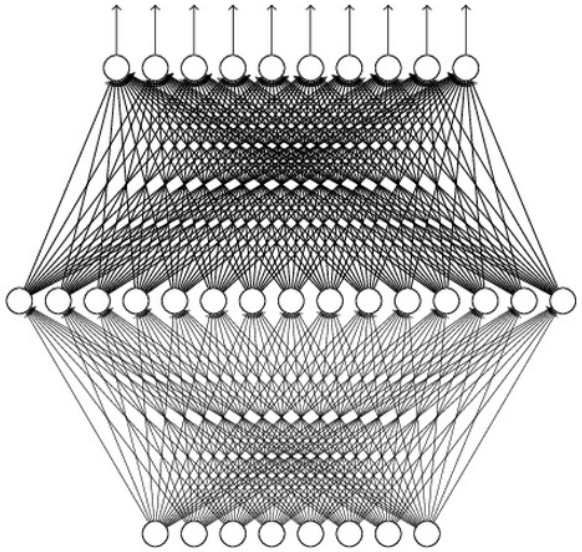
Universal Approximation Theorem

- Any continuous function f

$$f : R^N \rightarrow R^{\mathrm{M}}$$

- can be realized by a network with only one hidden layer

Why “deep” not “wide”?



How to frame the learning problem?

- The learning algorithm f is to map the input domain X into the output domain Y

$$f : X \rightarrow Y$$

- Input domain: word, word sequence, audio signal, click logs
- Output domain: single label, sequence tags, tree structure, probability distribution

Output Domain – Classification

- Sentiment Analysis

"I loved this product!" → +

"The screen is too small, design is awful" → -

- Speech Phoneme Recognition

→ /h/

- Handwriting Recognition

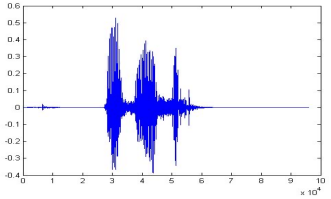
→ 2

Output Domain – Sequence Prediction

- POS Tagging

"I read the book" ⟶ I/PN read/VDB the/DT book/NN

- Speech Recognition



⟶ "the book"

- Machine Translation

"How are you doing today?" ⟶ "आज आप कैसे हैं?

Learning tasks are decided by the output domains!

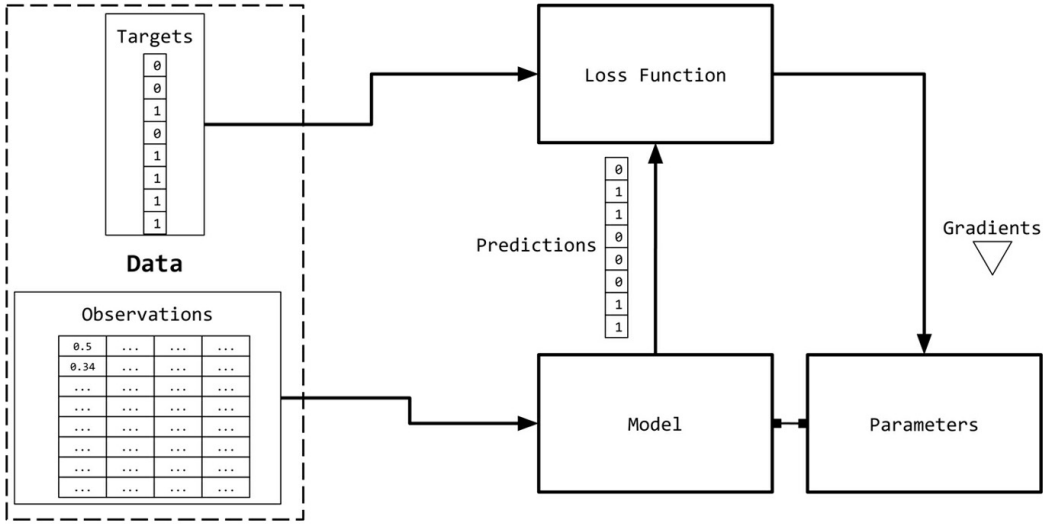
Input Domain –
How to aggregate information?

- Input: word sequence, image pixels, audio signal, click logs
- Properties: continuity, temporal, importance distribution
- Example
  - CNN (convolutional neural network): local connections, shared weights, pooling
    - AlexNet, VGGNet, etc.
  - RNN (recurrent neural network): temporal information

Network architectures should also consider the input domain properties!

Observation and Target Encoding

- Representations as numerical vectors.
- Learned from data
- Simple ways to represent:
  - One-hot encoding
  - Term frequency (TF) and bag-of-words
  - Term frequency – inverse document frequency (TF-IDF)



One-hot encoding

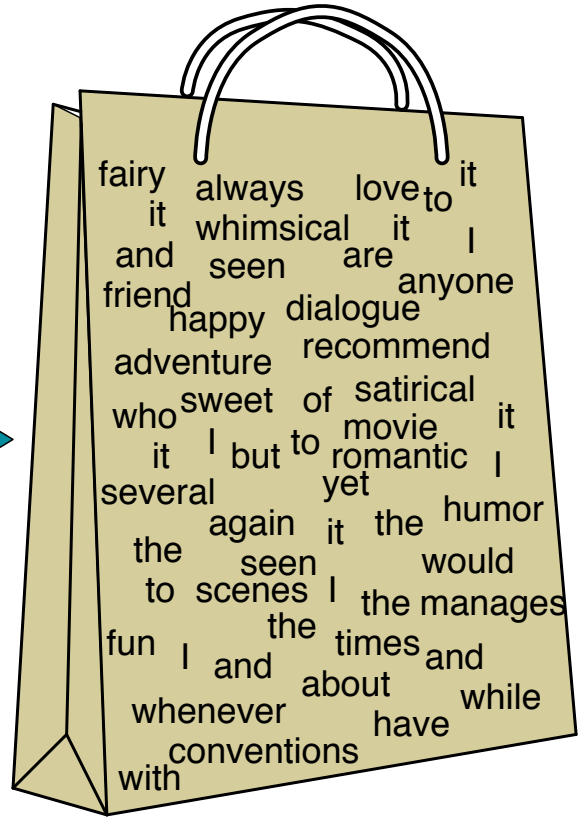
- Sentences: [Time flies like an arrow. Fruit flies like a banana]
- Vocabulary = { time, fruit, flies, like, a , an , arrow, banana}
- ”Time”: [1 0 0 0 0 0 0 0]
- “flies”: [0 0 1 0 0 0 0 0]
- “like”: [0 0 0 1 0 0 0 0]

	time	fruit	flies	like	a	an	arrow	banana
1_{time}	1	0	0	0	0	0	0	0
1_{fruit}	0	1	0	0	0	0	0	0
1_{flies}	0	0	1	0	0	0	0	0
1_{like}	0	0	0	1	0	0	0	0
1_{a}	0	0	0	0	1	0	0	0
1_{an}	0	0	0	0	0	1	0	0
1_{arrow}	0	0	0	0	0	0	1	0
1_{banana}	0	0	0	0	0	0	0	1

The binary encoding for “like a banana” would then be: [0, 0, 0, 1, 1, 0, 0, 1].

The Bag of Words Representation

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!



it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
…	…

The Bag of Words Representation

- Vocabulary = { time, fruit, flies, like, a , an , arrow, banana}

…

- ”Time flies like an arrow.”: [1 0 1 1 0 1 1 0]
- “Fruit flies like a banana.”: [0 1 1 1 1 0 0 1]

TF-IDF

- The IDF representation penalizes common tokens and rewards rare tokens in the vector representation.

$$IDF\,(w) = \log \frac{N}{n_w}$$

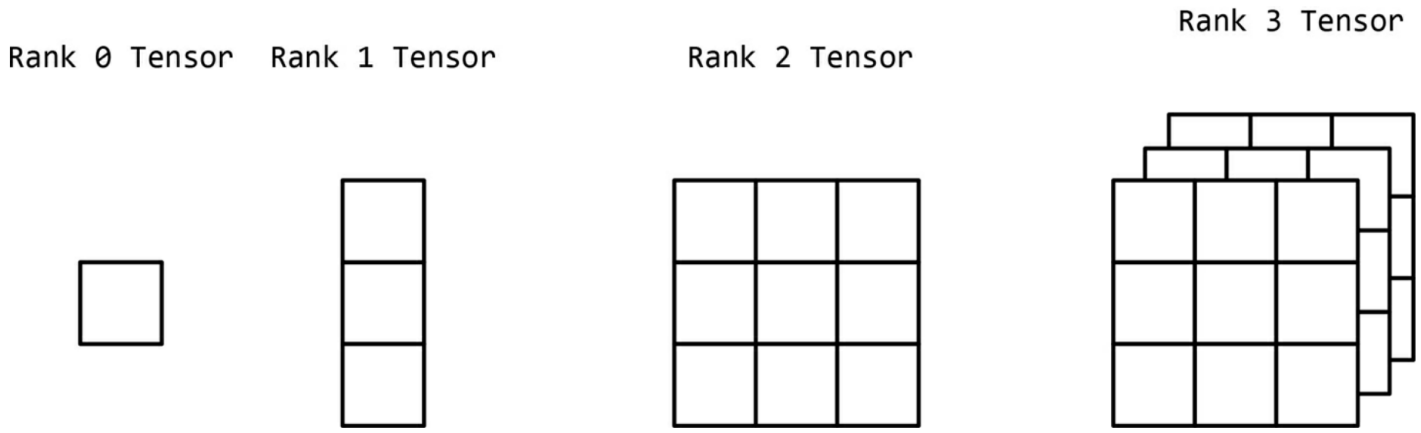
- n_w is the number of documents containing the word *w*
- *N* is the total number of documents.
- TF-IDF = product *TF(w) * IDF(w)*.

Outline for Today

- Continuing: What is deep learning?
- Preliminaries
  - PyTorch Basics and Linear Algebra
  - Calculus - Derivatives and Differentiation
- Readings:
  - Ch1 of NLP with PyTorch
  - Ch2 of Dive into Deep Learning -> The review chapter!

PyTorch Basics

- PyTorch is an optimized tensor manipulation library.
- It offers an array of packages for deep learning.
- A **tensor** is an algebraic object for representing multi-dimensional data.
- Vectors are first order tensors, etc.



Creating Tensors in PyTorch

```
[2] def describe(x):
        print("Type: {}".format(x.type()))
        print("Shape/size: {}".format(x.shape))
        print("Values: \n{}".format(x))
```

```
describe(torch.Tensor(2, 3))
```

```
Type: torch.FloatTensor
Shape/size: torch.Size([2, 3])
Values:
tensor([[2.4610e-36, 0.0000e+00, 3.7835e-44],
        [0.0000e+00,        nan, 0.0000e+00]])
```

Creating Tensors in PyTorch

- X=torch.zeros(2,3)
- X=torch.ones(2,3)
- X.fill_(5) # fills X with value 5
- X=torch.tensor([[1,2,3],[4,5,6]])
- X=torch.rand(2,3)
- X=torch.randn(2,3)
- Also from NumPy arrays

Tensor Operations

- Traditional programming language operations, such as, +,-,*,/.
- Functions such as add()
- Dimension-based operations
- Transpose

- Section will have a review!
- Also examples in the notebooks for Chapter 1 of NLP with PyTorch book
- https://github.com/joosthub/PyTorchNLPBook/tree/master/chapters/chapter_1

```python
a=torch.tensor([4, 3])
b=torch.tensor([2, 6])
print(a+b)
print(a-b)
print(a*b)
print(a // b)
print(a**b)
```

```
tensor([6, 9])
tensor([ 2, -3])
tensor([ 8, 18])
tensor([2, 0])
tensor([ 16, 729])
```

Reduction – Sum/Mean

- By default, invoking the sum function *reduces* a tensor along all its axes to a scalar.

```
a=torch.ones(3,4,5)
print(a.sum())
```

```
tensor(60.)
```

```
a=torch.ones(3,4,5,6)
print(a.sum())
```

```
tensor(360.)
```

```
[29] describe(a.mean())
```

```
Type: torch.FloatTensor
Shape/size: torch.Size([])
Values:
1.0
```

Non-Reduction Sum/Mean

- Sometimes it can be useful to keep the number of axes unchanged when invoking sum or mean by setting keepdims=True.

```
[17] a=torch.ones(3,4)
     print(a.sum())

     tensor(12.)

[18] sum_A = a.sum(axis=1, keepdims=True)
     print(sum_A)

     tensor([[4.],
             [4.],
             [4.]])
```

```
describe(a.sum(axis=0, keepdim=True))

Type: torch.FloatTensor
Shape/size: torch.Size([1, 4])
Values:
tensor([[3., 3., 3., 3.]])
```

```
describe(a.mean(axis=0, keepdim=True))

Type: torch.FloatTensor
Shape/size: torch.Size([1, 4])
Values:
tensor([[1., 1., 1., 1.]])
```

Tensor Operations - Product

- The size and shape of input arguments do not change for some operations. For example, elementwise product of two matrices (also called Hadamard product), with math notation, $\odot$.

$$\mathbf{A} \odot \mathbf{B} = \begin{bmatrix} a_{11}b_{11} & a_{12}b_{12} & \dots & a_{1n}b_{1n} \\ a_{21}b_{21} & a_{22}b_{22} & \dots & a_{2n}b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}b_{m1} & a_{m2}b_{m2} & \dots & a_{mn}b_{mn} \end{bmatrix}.$$

Dot Product

- Given two vectors $\mathbf{x},\mathbf{y}\in\mathbb{R}^d$, their *dot product* $\mathbf{x}^\top\mathbf{y}$ (or $\langle\mathbf{x},\mathbf{y}\rangle$) is a sum over the products of the elements at the same position:

```
[26] a=torch.Tensor([2,2,2])
     b=torch.ones(3)
     print(a,b)
     print(np.dot(a,b))
```

```
tensor([2., 2., 2.]) tensor([1., 1., 1.])
6.0
```

$$\mathbf{x}^\top\mathbf{y} = \sum_{i=1}^{d} x_i y_i.$$

- Weighted sum could be expressed as a dot product: $\mathbf{x}^\top\mathbf{w}$
- After normalizing two vectors to have the unit length, the dot products express the cosine of the angle between them.

Matrix-Matrix Multiplication

- Two matrices, A ∈ $\mathbb{R}^{n\text{x}k}$ and B ∈ $\mathbb{R}^{k\text{x}m}$

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1k} \\ a_{21} & a_{22} & \cdots & a_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nk} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1m} \\ b_{21} & b_{22} & \cdots & b_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ b_{k1} & b_{k2} & \cdots & b_{km} \end{bmatrix}.$$

- They can also be represented as:

$$\mathbf{A} = \begin{bmatrix} \mathbf{a}_1^\top \\ \mathbf{a}_2^\top \\ \vdots \\ \mathbf{a}_n^\top \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} \mathbf{b}_1 & \mathbf{b}_2 & \cdots & \mathbf{b}_m \end{bmatrix}.$$

- Their product is:

$$\mathbf{C} = \mathbf{AB} = \begin{bmatrix} \mathbf{a}_1^\top \\ \mathbf{a}_2^\top \\ \vdots \\ \mathbf{a}_n^\top \end{bmatrix} \begin{bmatrix} \mathbf{b}_1 & \mathbf{b}_2 & \cdots & \mathbf{b}_m \end{bmatrix} = \begin{bmatrix} \mathbf{a}_1^\top \mathbf{b}_1 & \mathbf{a}_1^\top \mathbf{b}_2 & \cdots & \mathbf{a}_1^\top \mathbf{b}_m \\ \mathbf{a}_2^\top \mathbf{b}_1 & \mathbf{a}_2^\top \mathbf{b}_2 & \cdots & \mathbf{a}_2^\top \mathbf{b}_m \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{a}_n^\top \mathbf{b}_1 & \mathbf{a}_n^\top \mathbf{b}_2 & \cdots & \mathbf{a}_n^\top \mathbf{b}_m \end{bmatrix}.$$

Matrix-Matrix Multiplication

```python
import numpy as np
a=torch.Tensor([[1,2,3]])
b=torch.ones(3,1)
print("a:")
describe(a)
print("b:")
describe(b)
print("a.b:", torch.mm(a,b))
print("b.a:", torch.mm(b,a))
```

```
a:
Type: torch.FloatTensor
Shape/size: torch.Size([1, 3])
Values:
tensor([[1., 2., 3.]])
b:
Type: torch.FloatTensor
Shape/size: torch.Size([3, 1])
Values:
tensor([[1.],
        [1.],
        [1.]])
a.b: tensor([[6.]])
b.a: tensor([[1., 2., 3.],
        [1., 2., 3.],
        [1., 2., 3.]])
```

a= [1 2 3]

$$b = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

Norms

- Informally norm of a vector tells us how big it is.
- In linear algebra, a vector norm is a function f that maps a vector to a scalar, satisfying a handful of properties:

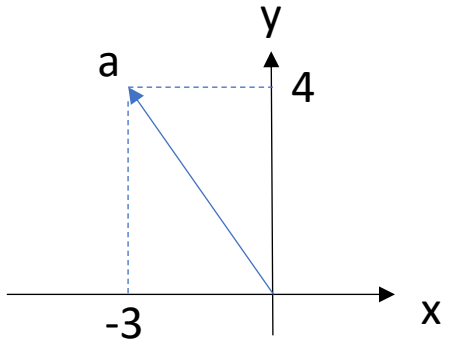
1. $\|\alpha A\| = |\alpha| \|A\|$
2. $\|A + B\| \leq \|A\| + \|B\|$
3. $\|A\| \geq 0$
4. If $\forall i, j, a_{ij} = 0$, then $\|A\| = 0$

- Euclidean distance is a norm, specifically it is the ℓ_2 norm

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2},$$

```
[49] a=torch.Tensor([-3,4])
     print(np.linalg.norm(a))
```

```
5.0
```



Colab Link for Examples

- https://colab.research.google.com/drive/1uWjnouLBXYC95QZldoUPqZIOdiuGA4Ak?usp=sharing

Outline for Today

- Continuing: What is deep learning?
- Preliminaries
  - PyTorch Basics and Linear Algebra
- Calculus - Derivatives and Differentiation
- NLP Basics

- Readings:
  - Ch1 of NLP with PyTorch
  - Ch2 of Dive into Deep Learning -> The review chapter!

Derivatives and Differentiation

- Training deep learning models: updating them successively so that they get better and better as they see more data
- Getting better: minimizing a loss function
- A crucial step in nearly all deep learning optimization algorithms.
- Loss functions that are differentiable with respect to our model's parameters.

Derivatives and Differentiation (cont.)

- Suppose we have function $f:\mathbb{R}\rightarrow\mathbb{R}$ with scalar input and outputs.
- The *derivative* of f is defined as:

$$f'(x) = \lim_{h\to 0} \frac{f(x+h) - f(x)}{h}$$

- If $f'(a)$ exists, f is said to be *differentiable* at a.
- If f is differentiable at every number of an interval, then this function is differentiable on this interval.
- Alternative notations for derivatives:

$$f'(x) = y' = \frac{dy}{dx} = \frac{df}{dx} = \frac{d}{dx}f(x) = Df(x) = D_x f(x).$$

Rules for Differentiating Common Functions

- $DC = 0$ (C is a constant),
- $Dx^n = nx^{n-1}$ (the *power rule*, n is any real number),
- $De^x = e^x$,
- $D\ln(x) = 1/x$.

Rules for Differentiating Common Functions (cont.)

$$\frac{d}{dx}[Cf(x)] = C\frac{d}{dx}f(x)$$

Constant multiple rule

$$\frac{d}{dx}[f(x) + g(x)] = \frac{d}{dx}f(x) + \frac{d}{dx}g(x)$$

Sum rule

$$\frac{d}{dx}[f(x)g(x)] = f(x)\frac{d}{dx}[g(x)] + g(x)\frac{d}{dx}[f(x)]$$

Product rule

$$\frac{d}{dx}\left[\frac{f(x)}{g(x)}\right] = \frac{g(x)\frac{d}{dx}[f(x)] - f(x)\frac{d}{dx}[g(x)]}{[g(x)]^2}$$

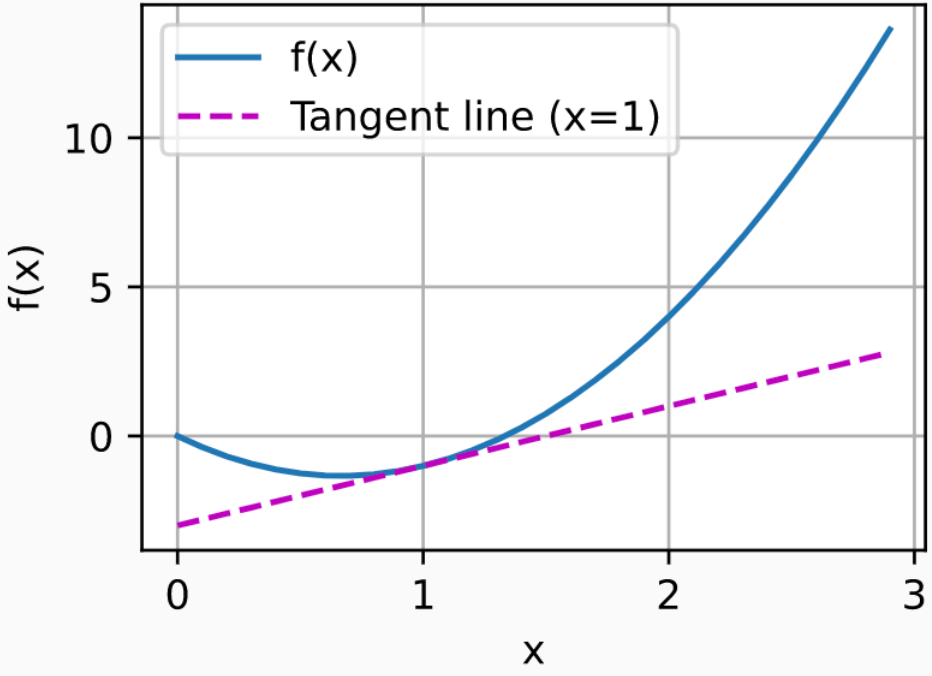
Quotient rule

Example

- Given function, $u = f(x) = 3x^2 - 4x$
- We can apply the previous set of rules to get its derivative:

$$u' = f'(x) = 3\frac{d}{dx}x^2 - 4\frac{d}{dx}x = 6x - 4$$

- For x=1, we have u'=2.
- This derivative is also the slope of the tangent line to the curve u=$f(x)$ when x=1.



Partial Derivatives

- If y is a function with n variables, i.e.,

$$y=f(x_1,x_2,\ldots,x_n)$$

- The **partial derivative** of y with respect to its ith parameter x_i is:

$$\frac{\partial y}{\partial x_i} = \lim_{h \to 0} \frac{f(x_1, \ldots, x_{i-1}, x_i + h, x_{i+1}, \ldots, x_n) - f(x_1, \ldots, x_i, \ldots, x_n)}{h}$$

Gradients

- For functions with multiple variables, we obtain the **gradient vector** of the function by concatenating partial derivatives of that function w.r.t. all its variables.
- For example, for $f:\mathbb{R}^n \rightarrow \mathbb{R}$ is a function with an n-dimensional vector $\mathbf{x}=[x_1,x_2,\ldots,x_n]^\top$ input and a scalar output. The gradient of the function $f(\mathbf{x})$ w.r.t. $\mathbf{x}$ is a vector of n partial derivatives:

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = \left[\frac{\partial f(\mathbf{x})}{\partial x_1}, \frac{\partial f(\mathbf{x})}{\partial x_2}, \ldots, \frac{\partial f(\mathbf{x})}{\partial x_n}\right]^\top$$

The Chain Rule

- Suppose that functions $y=f(u)$ and $u=g(x)$ are both differentiable, then,

$$\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx}$$

- More general scenario, where a differentiable function y has variables $u_1,u_2,\ldots,u_m$, where each differentiable function u_i has variables $x_1,x_2,\ldots,x_n$. Then, for any i=1,2,…,n,

$$\frac{dy}{dx_i} = \frac{dy}{du_1}\frac{du_1}{dx_i} + \frac{dy}{du_2}\frac{du_2}{dx_i} + \cdots + \frac{dy}{du_m}\frac{du_m}{dx_i}$$

Topics for Next Week

- NLP basic
- Getting ready for HomeWorks: Knowledge graphs and querying knowledge graphs
- Gradient Descent (and versions)
- Practical Tips
- Linear Neural Networks for Classification and Regression